Lecture 1

Task Environments

1. Fully Observable vs Partially Observable

Access to complete state of environment at each point in time

2. Deterministic vs Stochastic

Next state of environment is complete determined by current state and action executed by agent

Strategic: environment dependent on actions of other agents

3. Episodic vs Sequential

Agent's experience is divided into atomic "episodes" -> each episode consists of agent perceiving and then performing an action. Choice of action in each episode depends only on the episode itself

4. Static vs dynamic

Environment is unchanged while an agent is deliberating

Semi-dynamic: environment itself does not change with passage of time but agent's performance score does

5. Discrete vs Continuous

Limited number of distinct, clearly defined percepts and actions

6. Single agent vs multi-agent

Agent operating by itself in an environment

Lecture 2

Problem Formulation

E.g. Sudoku

1. States: All possible assignments of numbers to a 9x9 matrix
2. Initial State: Partially filled matrix
3. Goal State(s)/Test: Check if all constraints are satisfied
4. Actions: Fill matrix
5. Transition Model: Current state = filled matrix
6. Action Cost Function: 1

Data Structures

1. Queue (FIFO)
2. Priority Queue
3. Stack (LIFO)

Completeness: An algorithm is complete if, for every problem instance, it will find a solution if one exists. Conversely, an algorithm is incomplete if there exists at least one instance with a solution that the algorithm fails to find.

Optimality: An algorithm is optimal if, for every instance where it produces a solution, that solution is the best possible. In other words, if the algorithm outputs a solution, it is guaranteed to be optimal.

An algorithm can be both optimal and incomplete. This means that there exists at least one instance where the algorithm fails to find a solution, even though, when it does find a solution, that solution is guaranteed to be optimal.

Uninformed Search

When you have no idea how good a state is.

1. BFS: frontier - queue
2. DFS: frontier - stack
3. UCS: frontier - priority queue
4. Depth limited: limit search to depth l, backtrack when limit hit
5. Iterative Deepening: DLS but increase depth limit everytime

	Time	Space	Complete?	Optimal?
BFS	Exponential	Exponential	Yes if finite	Yes if cost is same everywhere
UCS	Exponential	Exponential	Yes if +ve cost and finite total cost	Yes if +ve cost
DFS	Exponential	Polynomial	No when depth is infinite	No, optimal soln may be shallower
DLS	Exponential	Polynomial	No if used with DFS	No if used with DFS
IDS	Exponential	Polynomial	Yes	Yes if cost same everywhere

Informed Search

Heuristic: estimate of optimal path cost from any state to goal state E.g. Straight line distance/Euclidean or Manhattan distance: steps to take from a grid square to another if only vertical & horizontal steps are allowed

A* Search: f(n) = g(n) + h(n); frontier - priority queue(f(n))

g(n): cost to reach n

h(n) estimated cost from n to goal

Time complexity: exponential (can be improved by h(n))

Space complexity: exponential

Complete: Yes if edge costs are +ve and branching factor finite

Optimal: depends on heuristic

Heuristic

Admissible: h(n) is admissible if for every node n, $h(n) \le h^*(n)$, where $h^*(n)$ is the optimal path cost to reach the goal state from n. (Basically never over estimates cost to reach goal - optimistic estimate). If h(n) admissible, A* search without visited memory is optimal.

Consistent: h(n) is consistent if for every node n, every successor n' of n generated by any action a, $h(n) \le c(n,a,n') + h(n')$, and h(G) = 0. If h(n) is consistent, A* search with visited memory is optimal. If h(n) is consistent, it is also admissible.

Dominance: if h1(n) >= h2(n) for all n, h1 dominates h2 and is better for search is admissible.

Lecture 3

Local Search

State space: all possible configs or states of a system

Search space: subset of state space that will be explored

Problem Formulation

Eg N-Queens

1. States: config of queens on board
2. Initial State: random placement of queens
3. Goal Test (optional): no 2 queens threaten each other
4. Successor Func: move queen to diff row within same column

Eval funcn used to assess quality of state within given search space. May minimise/maximise this.

Hill Climbing (Steepest Ascent Search, Greedy Local Search)

Pick best among neighbours, repeat

State space landscape

Global maximum: overall highest pt (optimal soln)

Local maximum: pt higher than imm neighbours, may not be optimal

Shoulder: flat area, makes it difficult for algo to move past

Adversarial Search

Adversarial games: agents compete against each other

Problem Formulation

1. States
2. Initial State
3. Terminal States: game ends (win/lose/draw)
4. Actions
5. Transition
6. Utility Function: output value of state from perspective of our agent. We want to maximise the utility.

Minimax

Player A try to maximise outcome of game (our pov)

Player B try to minimise outcome of Player A

Time complexity: Exponential wrt max depth O(b^m)

Space complexity: Polynomial

Complete: Yes if tree is finite

Optimal: Yes against optimal opponent

Eval Func: assigns score to game state, indicates favorability

- If terminal state: use utility(state)
- Else use heuristic: estimates utility of state. Should be between a loss and a win. Admissibility and consistency properties do not apply

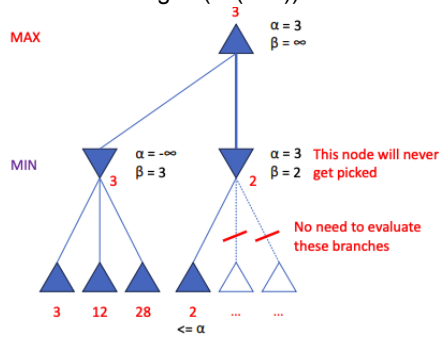
Alpha-beta pruning

Based on explored moves so far keep track of highest and lowest val the MAX player can obtain

Does not affect final result

Good move ordering improves effectiveness of pruning

Perfect ordering: O(b^(m/2))



Lecture 4

Types of machine learning

Supervised: Agent learns from labeled data (input & output pairs) to learn a mapping from inputs to outputs.

- Classification: predict a discrete label based on input (categorical value output eg spam detection)
- Regression: predict a continuous numerical value based on input (real number output eg house price)

Assume there is an underlying true rs between inputs x and labels y in dataset D. y = f*(x) + c. c is some noise or error term, accounts for randomness in data generation process.

Goal: find a function that best approx f*(x)

f*(x) ≈ h(x) = A(Dtrain)

Test hypothesis on test set

Measure error (if output of h is continuous val):

Absolute error = | y^ - y |

Squared error = (y^ - y)^2

Where y^ = h(x)

Mean squared error

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}^{(i)} - y^{(i)})^2 \quad \text{Where } \hat{y}^{(i)} = h(x^{(i)}).$$

Mean Absolute Error

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|\hat{y}^{(i)} - y^{(i)}| \quad \text{Where } \hat{y}^{(i)} = h(x^{(i)}).$$

Correctness/Accuracy

$$Accuracy = \frac{1}{N}\sum_{i=1}^{N}\mathbb{1}_{\hat{y}^{(i)} = y^{(i)}} \quad \text{Where } \hat{y}^{(i)} = h(x^{(i)}).$$

Confusion Matrix

Accuracy = (TP + TN) / (TP + FN + FP + TN)

Precision P = TP / (TP + FP)

Recall R = TP / (TP + FN)

F1 = 2 / (1/P + 1/R)

Decision Tree

Distinct decision tree with n boolean attributes = 2^(2^n))

Entropy: measure of randomness

$$I(P(v_1), \dots, P(v_k)) = -\sum_{i=1}^{k} P(v_i)\log_2 P(v_i)$$

Remainder: average entropy after dividing

$$remainder(A) = \sum_{i=1}^{v}\frac{p_i+n_i}{p+n} I\left(\frac{p_i}{p_i+n_i}, \frac{n_i}{p_i+n_i}\right)$$

Information gain: try to maximise

$$IG(A) = \boxed{I\left(\frac{p}{p+n}, \frac{n}{p+n}\right)} - \boxed{remainder(A)}$$

Entropy of this node *Entropy of children nodes*

Decision Tree Learning: Greedy top-down recursive algorithm

Pruning: min sample leaf / max depth tree

Lecture 5 (Linear Regression)

Dot product: $u^T v = [u_1 \quad u_2 \quad \dots \quad u_d]\begin{bmatrix} v_1 \\ v_2 \\ \dots \\ v_d \end{bmatrix} = \sum_{j=1}^{d} u_j v_j$

Linear Model: $h_w(x) = w^T x$

Feature Transformations

Feature Engineering:

- Polynomial Features: z = x^k
- Log Features: z = log(x)
- Exp Features: z = e^x

Feature Scaling:

- min-max scaling: $z_i = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$
- standardisation: $z_i = \frac{x_i - \mu_i}{\sigma_i}$ (transforms features to have mean 0 and sd 1)

Loss function / MSE: $J_{MSE}(w) = \frac{1}{2N}\sum_{i=1}^{N}\left(h_w(x^{(i)}) - y^{(i)}\right)^2$

Normal Equation (set derivative to 0, solve)

Minimum: $\frac{\partial J_{MSE}(w)}{\partial w_j} = 0 = \frac{1}{N}\sum_{i=1}^{N}\left(h_w(x^{(i)}) - y^{(i)}\right)x_j^{(i)}$

$$X^T(Xw - Y) = 0 \qquad \boxed{w = (X^T X)^{-1} X^T Y}$$

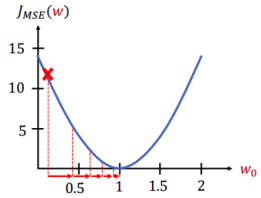
Gradient Descent

1. Start at some w
2. Update w with a step in opp direction of gradient

$$w_j \leftarrow w_j - \underbrace{\gamma}_{\text{Learning Rate}} \frac{\partial J(w_0, w_1, \dots)}{\partial w_j}.$$

y > 0 determines step size

3. Repeat until termination criterion satisfied



As it gets closer to a minimum,

- The magnitude of the **slope** becomes smaller
- The **step size** become smaller

With 2 parameters:

$$a = \frac{\partial J(w_0, w_1)}{\partial w_0} \qquad w_0 = w_0 - \gamma a$$

$$b = \frac{\partial J(w_0, w_1)}{\partial w_1} \qquad w_1 = w_1 - \gamma b$$

A convex func has a single global minimum = MSE loss func is convex for linear regression (even aft feature transformations)

Variations of gradient descent:

Batch: consider all training examples

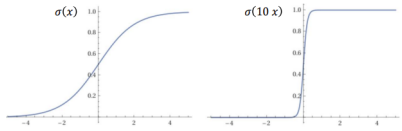
Mini-batch: consider a subset

Stochastic: select one random data point at a time

	Gradient descent	Normal Eqn
Choose learning rate	yes	no
Iterations	many	none
Large no of features d	No problem	Slow O(d^3)
Feature scaling	May be necessary	no
Constraints	-	X^TX needs to be invertible

Lecture 6 (Logistic Regression)

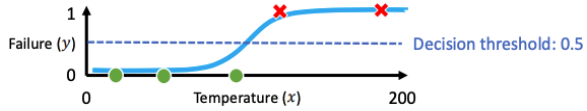
$\sigma(x) = \frac{1}{1 + e^{-x}}$ derivative: $\sigma'(x) = \sigma(x)(1 - \sigma(x))$



Logistic Model: $h_w(x) = \sigma(w^T x)$ output range: [0,1]

Output can be treated as probability of an input to be of class 1

Deciding whether input belongs to certain class: compare probability output to a decision threshold:



$$h_w(x) = \sigma(-100 + 1x_1)$$

Non-linearly separable: single linear decision boundary cannot effectively separate diff classes -> use feature transformation

Binary Cross Entropy (BCE): diff betw 2 probability vals

$$BCE(y, \hat{y}) = -y\log(\hat{y}) - (1 - y)\log(1 - \hat{y})$$

True val y and prediction y^ close: BCE loss small

BCE / logistic loss: $J_{BCE}(w) = \frac{1}{N}\sum_{i=1}^{N} BCE\left(y^{(i)}, h_w(x^{(i)})\right)$

Multi-Class Classification

- One-vs-one: C classes -> (C(C - 1))/2 classifiers (most votes)
- One-vs-rest: C classes -> C classifiers (pick highest prob)

Generalisation (Relevance, Noise, Balance/Representation)

Model Complexity (Low -> Underfitting, High -> Overfitting)

Simple truth -> simple model (high bias, low variance)

Complex truth -> complex model (low bias, high variance)

High complexity: ↑ data quantity, ↓ error on unseen data, error on UD gets closer to that of training data)

Hyperparameter Tuning Techniques

Grid Search (All possible combi of hyperparams), Random search, Local search